

UNIVERSIDAD TECNOLÓGICA NACIONAL

Instituto Nacional Superior del Profesorado Técnico - INSPT

Carrera: Informática Aplicada | 1 Año | Asignatura: Laboratorio

L A B O R A T O R I O

Estructuras de Control

Selección múltiple (Según) - Ciclos Repetitivos - La lógica es universal

Prof. Behringer Alejandro Jorge | Prof. Capia Juan Carlos

Ciclo Lectivo 2026 - Comisiones 1601 - 1602 - 1603 - 1604 - 1605

01

Selección múltiple

Estructura Segun / Hacer - evaluación de múltiples casos

02

Ciclo Repetir-Mientras

Hacer ... Mientras Que - se ejecuta al menos una vez (do/while)

03

Ciclo Para

Para ... Hasta ... Con Paso - número fijo de iteraciones (for)

04

Ciclo Mientras

Mientras ... Hacer - evalúa condición antes de entrar (while)

05

La lógica es universal

Un problema - tres dialectos: PSeInt - C - Python

¿Qué es?

Permite evaluar el valor de una variable y ejecutar un bloque diferente para cada caso posible. Es más limpio que varios Si-SiNo anidados.

¿Cuándo usarlo?

- ✓ Cuando se evalúa una sola variable
- ✓ Múltiples valores posibles y conocidos
- ✓ Más legible que Si-SiNo anidados
- ✗ No sirve para rangos -> usar Si

Sintaxis en PSeInt (estricta)

```
Segun <variable> Hacer
    valor1 : <instrucciones>;
    valor2, valor3 : <instr.>;
    ...
De Otro Modo:
    <instrucciones>;
FinSegun
```

NOTA: En sintaxis estricta los valores deben ser numéricos. Para usar caracteres personalizar el perfil (destildar Limitar Segun a numéricos).

El usuario ingresa 1, 2 o 3 y el programa indica la operación elegida:

```
Proceso menu_operaciones
  Definir opcion Como Entero;
  Escribir "Ingrese opcion:";
  Escribir " 1-Suma  2-Resta  3-Mult";
  Leer opcion;
  Segun opcion Hacer
    1 : Escribir "Elegiste: SUMA";
    2 : Escribir "Elegiste: RESTA";
    3 : Escribir "Elegiste: MULT";
  De Otro Modo:
    Escribir "Opcion invalida";
  FinSegun
FinProceso
```

Prueba de escritorio:

opcion	Salida
1	Elegiste: SUMA
2	Elegiste: RESTA
3	Elegiste: MULT
5	Opcion invalida

El bloque De Otro Modo actúa como el else: captura cualquier valor no contemplado en los casos anteriores.

Segun / Hacer

Ciclos Repetitivos

Como hacer que el programa repita acciones?

**Repetir
Mientras Que**

Al menos 1 vez

Para

N fijo de veces

Mientras

0 o mas veces

Ciclo Repetir ... Mientras Que (do / while)

Ciclos Repetitivos

Característica clave

El cuerpo se ejecuta SIEMPRE al menos una vez. La condición se evalúa al FINAL de cada iteración.

Sintaxis en PSeInt:

```
Repetir
    <instrucciones>;
Mientras Que <condicion>;
```

Ejemplo: contar del 1 al 10

```
Proceso contar
    Definir a Como Entero;
    a <- 0;
    Repetir
        a <- a + 1;
        Escribir a;
    Mientras Que a < 10;
FinProceso
```

Flujo de ejecución:

- > 1. Entra al bloque
- > 2. Ejecuta instrucciones
- > 3. Evalua condición al final
- > 4a. Verdadera -> vuelve al paso 2
- > 4b. Falsa -> sale del ciclo

Repetir...Mientras Que

Característica clave

Se usa cuando se conoce de antemano cuantas veces se repite. La variable de control se incrementa automáticamente.

Sintaxis en PSeInt:

```
Para <var> <- <inicio> Hasta <fin> Con Paso <n> Hacer  
    <instrucciones>;  
FinPara
```

Ejemplo: mostrar del 0 al 9

```
Proceso contar_para  
    Definir a Como Entero;  
    Para a <- 0 Hasta 9 Con Paso 1 Hacer  
        Escribir a;  
    FinPara  
FinProceso
```

Cuando usarlo?

- ✓ Cuando el nro. de repeticiones es conocido
- ✓ Recorrer vectores o arreglos
- ✓ Generar secuencias numericas
- ✗ Cuando no sabes cuantas veces -> Mientras

Para / for

Característica clave

La condicion se evalua ANTES de entrar. Si es falsa desde el inicio, el cuerpo nunca se ejecuta (0 iteraciones posibles).

Sintaxis en PSeInt:

```
Mientras <condicion> Hacer
    <instrucciones>;
FinMientras
```

Ejemplo: contar mientras a < 10

```
Proceso contar_mientras
    Definir a Como Entero;
    a <- 0;
    Mientras a < 10 Hacer
        a <- a + 1;
        Escribir a;
    FinMientras
FinProceso
```

Diferencia clave con Hacer-Mientras Que:

- > Mientras evalua la condicion ANTES de entrar.
- > Puede ejecutarse 0 veces si la condicion es falsa.
- > Hacer evalua DESPUES: siempre entra al menos 1 vez.
- > Elegir Mientras cuando no estas seguro de si debe entrar.

Comparativa: Cuando uso cada ciclo?

Ciclos Repetitivos

Repetir-Mientras Que

Cuando?

Cuando el bloque SIEMPRE debe ejecutarse al menos una vez.

Ejemplo:

Menu que pide opcion valida al usuario.

Evalua condicion
DESPUES del bloque

Para

Cuando?

Cuando conoces de antemano cuantas veces se repite.

Ejemplo:

Recorrer un vector de 10 elementos.

Contador
automatico

Mientras

Cuando?

Cuando no sabes si el bloque debe ejecutarse o no.

Ejemplo:

Leer datos hasta que el usuario ingrese 0.

Evalua condicion
ANTES de entrar

"La logica es universal, la sintaxis es solo el dialecto"

El mismo algoritmo en tres lenguajes - el problema no cambio, solo la forma de escribirlo:

PSeInt

```
// Sumar del 1 al 10
Proceso sumar
  Definir i, s Como Entero;
  s <- 0;
  Para i <- 1 Hasta 10 Hacer
    s <- s + i;
  FinPara
  Escribir s;
FinProceso
```

C

```
// Sumar del 1 al 10
#include <stdio.h>
int main() {
  int i, s = 0;
  for (i=1; i<=10; i++)
  {
    s = s + i;
  }
  printf("%d", s);
  return 0;
}
```

Python

```
# Sumar del 1 al 10
s = 0
for i in range(1, 11):

    s = s + i

print(s)
```

Los tres programas calculan lo mismo: $1+2+3+...+10 = 55$ | La logica es identica, solo cambia el idioma

Hasta la proxima clase!

Resumen de hoy

- > Segun -> seleccion multiple sobre una variable
- > Repetir-Mientras Que -> cuerpo se ejecuta siempre al menos 1 vez
- > Para -> numero de repeticiones conocido de antemano
- > Mientras -> condicion evaluada antes de entrar al bloque

★ *"La logica es universal, la sintaxis es solo el dialecto"*

Prof. Behringer Alejandro Jorge | Prof. Capia Juan Carlos

UTN INSPT - Laboratorio - Ciclo Lectivo 2026